

AVP 세미나 5주차

Mingyu Choi

Architecture and Compiler for Embedded System LAB.

School of Electronics Engineering, KNU, KOREA

2026-07-16



목차

- 무작정 따라하는 ROS2 응용 과정
- HL FMA

Package와 Node 만들기

- `mkdir -p ~/ros2_study/src`

```
admin2@admin:~/ros2_study$ ls
src
admin2@admin:~/ros2_study$ colon build
colon: command not found
admin2@admin:~/ros2_study$ ls
src
admin2@admin:~/ros2_study$ sudo apt install python3-colcon-common-extensions
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3-colcon-common-extensions is already the newest version (0.3.0-100).
0 upgraded, 0 newly installed, 0 to remove and 423 not upgraded.
admin2@admin:~/ros2_study$ colcon build

Summary: 0 packages finished [3.94s]
admin2@admin:~/ros2_study$ ls
build install log src
admin2@admin:~/ros2_study$
```

mkdir 이후 ls 시 src폴더만 존재하지만 colcon build 이후
ls 시 build / install / log / src 폴더가 생성된다.

Package와 Node 만들기

- `ros2 pkg create --build-type ament_python --node-name my_first_node my_first_package`

```
admin2@admin:~/ros2_study/src$ tree
locales-launch: Data of en_US locale not found, ge
locales-launch: Data of ko_KR locale not found, ge
├── my_first_package
│   ├── my_first_package
│   │   ├── __init__.py
│   │   └── my_first_node.py
│   ├── package.xml
│   ├── resource
│   │   └── my_first_package
│   ├── setup.cfg
│   ├── setup.py
│   └── test
│       ├── test_copyright.py
│       ├── test_flake8.py
│       └── test_pep257.py
5 directories, 9 files
```

- **ament_python** : python 문법 사용
- **--node-name my_first_node** : package 내에 my_first_node도 생성

-> package가 잘 생성된 것을 볼 수 있다.

Package와 Node 만들기

- Ros2 run my_first_package my_first_node

```
admin2@admin:~/ros2_study$ ls install/  
COLCON_IGNORE          local_setup.zsh  
local_setup.bash       my_first_package  
local_setup.ps1        setup.bash  
local_setup.sh          setup.ps1  
_local_setup_util_ps1.py setup.sh  
_local_setup_util_sh.py setup.zsh
```

Install 폴더 내부 디렉토리들을 보면 local_setup.bash라는 파일이 있다.

노드를 실행하고 tap키를 먹이기 위해서는 이 파일을 터미널에서 불러와야 한다.

Package와 Node 만들기

- source ./install/local_setup.bash
➔ 워크스페이스 활성화

```
admin2@admin:~/ros2_study$ ros2 run my_first_package my_first_node
Hi from my_first_package.
```

```
my_first_node.py X
my_first_package > my_first_package > my_first_node.py > ...
1 def main():
2     print('Hi from my_first_package.')
3
4
5 if __name__ == '__main__':
6     main()
```

Setup.bash vs local_setup.bash

-> local_setup.bash는 그 워크스페이스에 있는 패키지들만 등록하고, setup.bash는 하위워크스페이스에 있는 setup을 순서대로 sourcing한다. 다음 마지막에 자기 워크스페이스의 local_setup.bash를 부른다.

메시지 정의 및 토픽과 서비스 다루기

- `ros2 pkg create --build-type ament_cmake my_first_package_msgs`

→ src 디렉토리 내에서 my_first_package 패키지에서 사용할 메시지를 정의

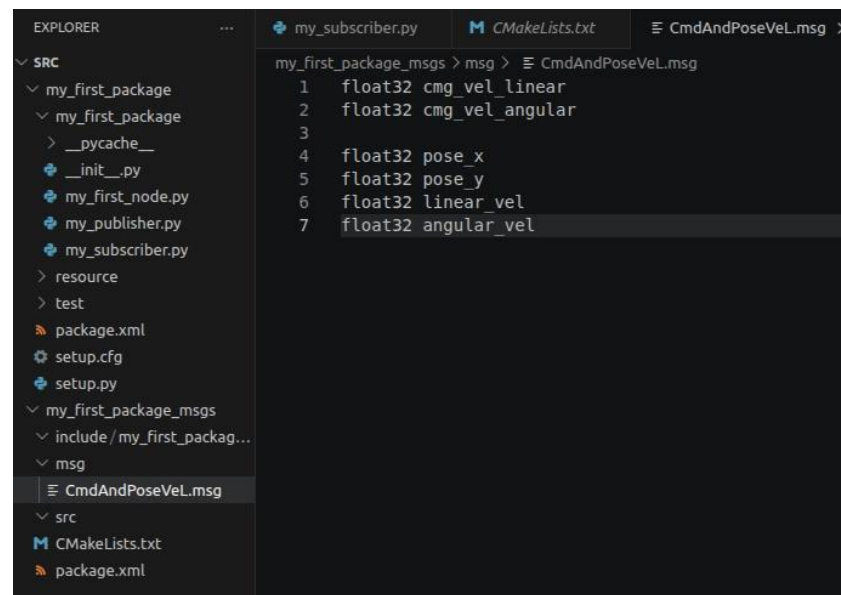
```
admin2@admin:~/ros2_study/src$ tree
.
├── my_first_package
│   ├── my_first_package
│   │   ├── __init__.py
│   │   ├── my_first_node.py
│   │   ├── my_publisher.py
│   │   ├── my_subscriber.py
│   │   ├── __pycache__
│   │   └── my_subscriber.cpython-310.pyc
│   ├── package.xml
│   ├── resource
│   │   └── my_first_package
│   ├── setup.cfg
│   ├── setup.py
│   └── test
│       ├── test_copyright.py
│       ├── test_flake8.py
│       └── test_pep257.py
├── my_first_package_msgs
│   ├── CMakeLists.txt
│   ├── include
│   │   └── my_first_package_msgs
│   ├── package.xml
│   └── src
└── 9 directories, 14 files
```

-> Tree로 my_first_package_msgs가 생성됨

메시지 정의 및 토픽과 서비스 다루기

• CmdAndPoseVelL.msg 파일 작성

- cmd_vel_linear : 명령한 직진 속도
- cmd_vel_angular : 명령한 회전 속도
➔ 발행자 cmd_vel이 내보낸 값
- pose_x : 거북이의 실제 x 좌표
- pose_y : 거북이의 실제 y 좌표
- linear_vel : 거북이의 실제 직진 속도
- angular_vel : 거북이의 실제 회전 속도
-> Turtlesim이 알려준 값



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a directory structure with a file named `CmdAndPoseVelL.msg` selected. The code editor shows the content of this file, which defines a message type with the following fields:

```
1 float32 cmd_vel_linear
2 float32 cmd_vel_angular
3
4 float32 pose_x
5 float32 pose_y
6 float32 linear_vel
7 float32 angular_vel
```


메시지 정의 및 토픽과 서비스 다루기

- CMakeLists.txt에서 해당 코드 작성

```
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/CmdAndPoseVel.msg"
)
```

➔ 빌드 시 CmdAndPoseVel.msg 파일을 C++/Python에서 쓸 수 있는 메시지 타입으로 만들어준다.

- Package.xml에서 해당 코드 작성

```
<build_depend>rosidl_default_generators</build_depend>
<exec_depend>rosidl_default_runtime</exec_depend>
<member_of_group>rosidl_interface_packages</member_of_group>
```

-> CmdAndPoseVel.msg를 만들려면 어떤 도구가 필요한지를 선언

메시지 정의 및 토픽과 서비스 다루기

- 결과

```
admin2@admin:~/ros2_study$ ros2 interface show my_firs
t_package_msgs/msg/CmdAndPoseVel
float32 cmd_vel_linear
float32 cmd_vel_angular

float32 pose_x
float32 pose_y
float32 linear_vel
float32 angular_vel
```

➔ 메시지 타입을 정의하고 빌드하고 등록하는 것까지 잘 된 것을 확인

HL FMA

- ROI image는 검출하는데 조향을 못하는 문제
조향이 생성되는 실제 경로 :

lane_info_extractor → path_planner → motion_planner

- 원인 1 : Look-ahead 부재

lane_info_extractor_node : 타겟점 y는 range(5,155,50)

→ Y = 5, 55, 105 딱 3개만 본다

Path_planner_node : 여기에 CAR_CENTER_POINT = (320,179)를 붙이고 y를 오름차순 정렬

정렬 후 점들 : (x5, 5) -> (x55,55) -> (x105,105) -> (320, 179)

x5 : 가장 먼 곳 / 320 : 범퍼(고정 앵커)

이 4점들을 CubicSpline으로 보간하여 y = 5~179 사이를 100점으로 리샘플 (인덱스 간격은 $(179-5)/99 = 1.76\text{px}$)

HL FMA

최종 경로 배열(motion_planner_node) :

```
target_slope =  
DMFL.calculate_slope_between_points(self.path_data[-10])  
- path_data[-1] = 인덱스 99 -> y=179 (범퍼)  
- path_data[-10] = 인덱스 90 -> y=163 (범퍼에서 겨우 9칸,  
약 16px 위)
```

⇒조향각을 범퍼 바로 앞 16px 짧은 조각의 기울기로 계산한다.
(보통 웹페이지 기본 한 글자 높이가 16px)

HL FMA

- ROI 이미지는 여전히 잘 떴고 문제 x
- 그러나 조향 가변저항이 고장/오정렬 상태 확인

오른쪽도 다시 1023. 이제 확정입니다. 반복 측정으로 나온 결과:

바퀴 위치	POT 값
왼쪽 끝	1023
직진	828
오른쪽 끝	1023
쓸어볼 때 중간 최소	596

● 결론: 조향 가변저항(POT)이 고장/오정렬 상태입니다

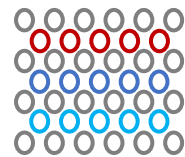
✓ 소프트웨어 - 전부 정상 (우리가 고침)

항목	상태
best.pt 모델 (차선 검출)	✓ 정상 (/detections lane 검출)
ROI 이미지 발행	✓ 정상 (/roi_image ~12Hz)
조향 p제어 (곡률 비례 ± 7)	✓ 확인 (3-4-5-7)
커브 감속	✓ 확인 (100-80)
조향 부호	✓ STEERING_SIGN=1 확정
import 이름 / pyc 트릭 제거	✓ 정리·빌드 완료

-> 왼쪽 끝과 오른쪽 끝이 같은 값이 측정 => 한 쪽으로만 조향

Q & A

Thank you for your attention



Architecture and Compiler
for Embedded Systems Lab.

Graduate School of Electronic and Electrical Engineering, KNU
ACE Lab. (hw051656@gmail.com)